

HOLIDAYHOMES

SAAS PLATFORM

Security Architecture Documentation

Version: 1.0

Date: May 31, 2026

Classification: Internal — Confidential

Target: UAE Holiday Homes Property Management



SECURITY DOCUMENT

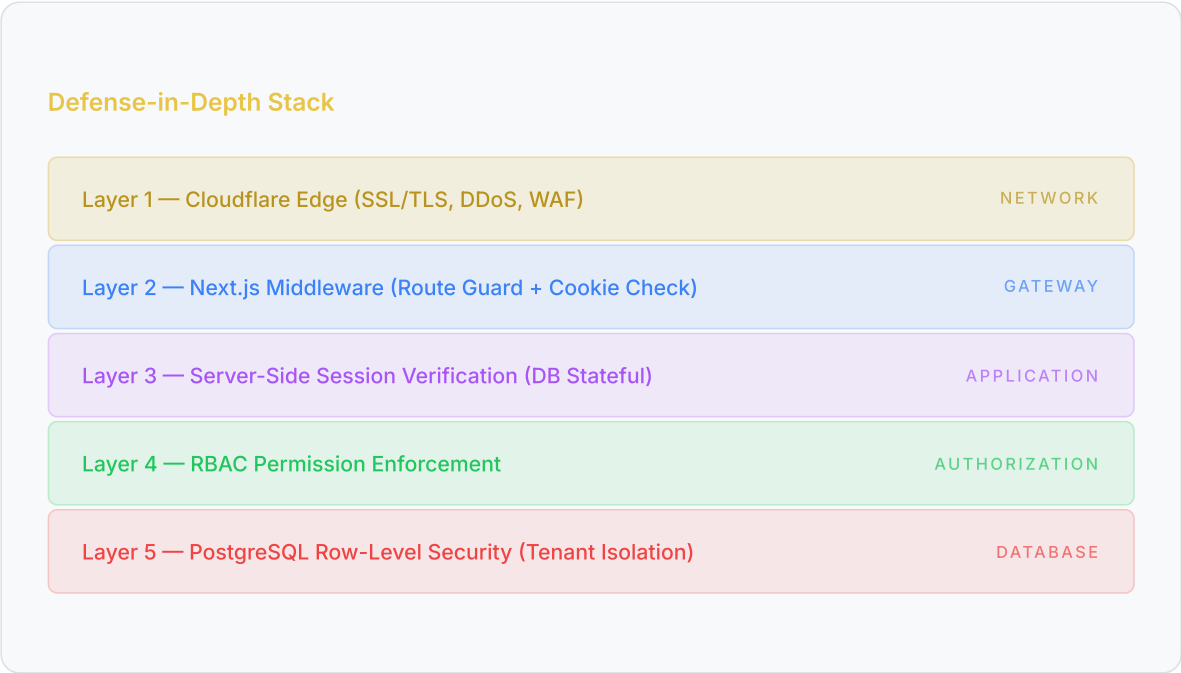
CONTENTS

Table of Contents

- 01 Security Architecture Overview
- 02 Authentication System
- 03 Multi-Factor Authentication (MFA)
- 04 Session Management
- 05 Role-Based Access Control (RBAC)
- 06 Multi-Tenancy & Data Isolation
- 07 Row-Level Security (RLS)
- 08 Route & Middleware Protection
- 09 Data Encryption & Hashing
- 10 Audit Logging
- 11 Cookie & Transport Security
- 12 Environment Separation

Security Architecture Overview

The Holiday Homes SaaS platform implements a defense-in-depth security model with multiple overlapping layers. Each layer provides independent protection — if one fails, subsequent layers prevent unauthorized access.



Security Technologies Used

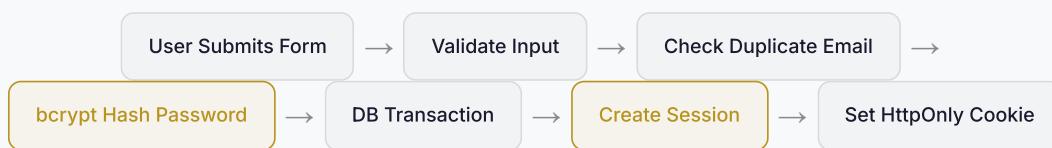
CONCERN	TECHNOLOGY	STANDARD
Password Hashing	bcryptjs (10 salt rounds)	OWASP Recommended
Session Tokens	crypto.randomBytes(32) → SHA-256	256-bit entropy
MFA Secret Encryption	AES-256-GCM with auth tag	NIST SP 800-38D
TOTP Verification	RFC 6238 (otplib)	30-second window
Tenant Isolation	PostgreSQL RLS Policies	Row-Level Security
Transport	TLS 1.3 via Cloudflare	HTTPS Only
Cookie Security	HttpOnly, Secure, SameSite=Strict	OWASP Cookie Guide

SECTION 02

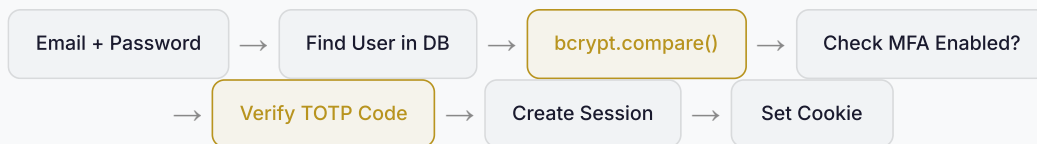
Authentication System

Custom-built authentication (no third-party auth providers). Credentials-based login with server-side stateful sessions stored in PostgreSQL.

Registration Flow



Login Flow



Password Storage

```
// password.ts - bcryptjs with 10 salt rounds
import bcrypt from 'bcryptjs';

const SALT_ROUNDS = 10;

export async function hashPassword(password: string): Promise<string> {
  return bcrypt.hash(password, SALT_ROUNDS);
}

export async function verifyPassword(password: string, hash: string): Promise<boolean> {
  return bcrypt.compare(password, hash);
}
```

✓ **Why bcrypt?** — Adaptive cost function makes brute-force attacks computationally expensive. 10 salt rounds ≈ ~100ms per hash, sufficient for production use while resistant to GPU attacks.

Security Decisions

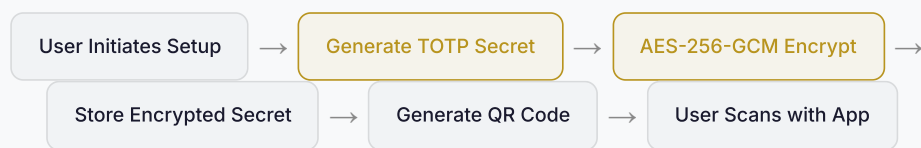
- **No plaintext password storage** — only bcrypt hashes stored in the `passwordHash` column.
- **Constant-time comparison** — `bcrypt.compare()` is inherently timing-safe.
- **Generic error messages** — login returns "Invalid email or password" regardless of whether the email or password is wrong (prevents user enumeration).
- **Account deactivation check** — `user.isActive` is verified before password comparison.

SECTION 03

Multi-Factor Authentication (MFA)

Time-based One-Time Password (TOTP) per RFC 6238. Compatible with Google Authenticator, Authy, and 1Password. MFA secrets are encrypted at rest using AES-256-GCM.

MFA Setup Flow



MFA Verification Flow



Secret Encryption Implementation

```
// mfa.ts - AES-256-GCM with authentication tag
const ALGORITHM = 'aes-256-gcm';
const IV_LENGTH = 12; // 96-bit IV per NIST recommendation

function encryptSecret(secret: string): string {
  const key = crypto.createHash('sha256').update(ENCRYPTION_KEY).digest();
  const iv = crypto.randomBytes(IV_LENGTH);
  const cipher = crypto.createCipheriv(ALGORITHM, key, iv);

  // Output format: iv_hex:auth_tag_hex:ciphertext_hex
  return `${iv.toString('hex')}:${cipher.getAuthTag().toString('hex')}:${cipher.update(secret).toString('hex')}`;
}
```



Encryption Key: Derived from `MFA_ENCRYPTION_KEY` environment variable via SHA-256. The key is *never* stored in source code or committed to version control.

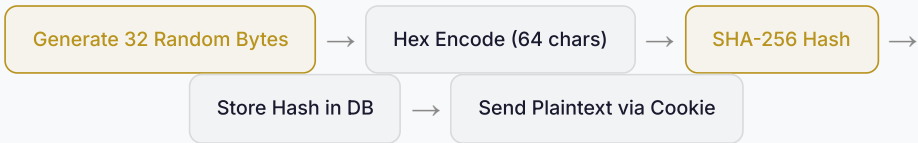
MFA Data Storage

COLUMN	CONTENT	ENCRYPTION
mfaSecret	TOTP secret key	AES-256-GCM (encrypted at rest)
isMfaEnabled	Boolean flag	Plaintext (non-sensitive)
backupCodes	JSON array of recovery codes	JSON (planned: hashed)

Session Management

Stateful, server-side sessions stored in PostgreSQL. No JWTs — tokens are opaque random strings whose SHA-256 hashes are stored in the database.

Session Token Lifecycle



```
// session.ts
function generateToken(): string {
  return crypto.randomBytes(32).toString('hex'); // 256-bit entropy
}

function hashToken(token: string): string {
  return crypto.createHash('sha256').update(token).digest('hex');
}
```

Session Database Record

FIELD	TYPE	PURPOSE
tokenHash	String (unique)	SHA-256 of the plaintext token — database never sees the plaintext
userId	UUID (FK)	The authenticated user
organizationId	UUID (FK, nullable)	Active tenant context for this session
ipAddress	inet	Client IP at session creation (audit/anomaly detection)
userAgent	String	Browser/device fingerprint
expiresAt	Timestamptz	7-day TTL — expired sessions are auto-revoked

⚠ **Why hash session tokens?** — If the database is compromised (SQL injection, backup leak), attackers cannot reconstruct valid session cookies from the stored hashes. This follows the

"tokens as secrets" pattern from OWASP Session Management Guidelines.

Session Verification on Every Request

```
// Every protected API route / server component calls:
const session = await verifySession(sessionToken);

// Verification checks:
// 1. Hash the cookie token → lookup in sessions table
// 2. If not found → return null (invalid token)
// 3. If expired → delete session row, return null
// 4. Load user + org + role + permissions → return full context
```

Session Revocation

- **Logout:** Session row is deleted from database + cookie is expired (set to epoch 0)
- **Expired:** Sessions past their 7-day TTL are deleted on next verification attempt
- **Deactivated User:** `user.isActive = false` prevents login; existing sessions continue until TTL but role check returns `guest`

Role-Based Access Control (RBAC)

Fine-grained permission system with wildcard support. Roles are scoped per-organization, allowing each tenant to customize access levels independently.

System Roles & Permission Matrix

ROLE	PROPERTIES	BOOKINGS	TASKS	MAINTENANCE	ACCOUNTING	WHATSAPP
Super Admin	★ ALL	★ ALL	★ ALL	★ ALL	★ ALL	★ ALL
Org Admin	★ ALL	★ ALL	★ ALL	★ ALL	★ ALL	★ ALL
Property Manager	✓ CRUD + Financial	✓ CRUD + Sync	✓ CRUD + Complete	✓ CRUD + Approve	X None	✓ Chat + Send
Accountant	✓ Read + Financial	✓ Read	X None	X None	✓ Full Access	X None
Coordinator	✓ Read	X None	✓ CRUD + Complete	✓ CRUD	X None	✓ Send
Owner	✓ Portal View	X None	X None	✓ Approve Quotes	✓ View Statements	X None
Vendor	X None	X None	X None	✓ Work Orders	✓ Submit Invoices	X None

Permission Enforcement Code

```
// permissions.ts - Server-side enforcement
export async function hasPermission(requiredPermission: string): Promise<boolean> {
  const permissions = getPermissionsFromSession();

  // Wildcard overrides for admins
  if (permissions.includes('*') || permissions.includes('admin.*')) {
    return true;
  }
}
```

```
return permissions.includes(requiredPermission);  
}  
  
// Usage in API routes:  
await requirePermission('accounting.manage_ledger');  
// Throws 403 Forbidden if user lacks this permission
```

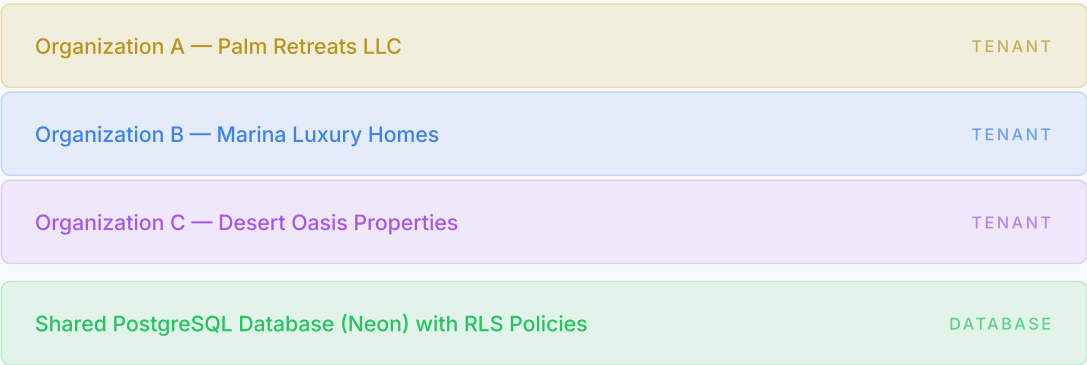
Permission Format

- `*` — Wildcard: full access to everything (Super Admin / Org Admin)
- `properties.create` — Resource-level: `{resource}.{action}`
- `owner_portal.view_statements` — Portal-scoped: limits access to portal features only

Multi-Tenancy & Data Isolation

Every data record belongs to an Organization. Isolation is enforced at the database level using PostgreSQL Row-Level Security, making cross-tenant data leakage impossible even if application code has bugs.

Tenant Architecture



 **Zero Trust Between Tenants:** Even if application code mistakenly omits a `WHERE organizationId = ...` filter, the PostgreSQL RLS policy will silently filter out rows belonging to other tenants. This is a database-level guarantee independent of application correctness.

All Tenant-Isolated Tables (34 total)

DOMAIN	TABLES
Auth & Admin	<code>organizations</code> , <code>organization_members</code> , <code>roles</code> , <code>audit_logs</code>
Properties	<code>properties</code> , <code>property_photos</code> , <code>property_documents</code> , <code>property_manager_assignments</code> , <code>owner_properties</code>
CRM	<code>contacts</code> , <code>vendor_categories</code>
Bookings	<code>bookings</code> , <code>uplisting_sync_logs</code>
Operations	<code>tasks</code> , <code>task_comments</code> , <code>task_attachments</code> , <code>maintenance_requests</code> , <code>maintenance_quotes</code> , <code>work_orders</code> , <code>claims</code> , <code>claim_evidences</code>
Accounting	<code>chart_of_accounts</code> , <code>journal_entries</code> , <code>journal_lines</code> , <code>invoices</code> , <code>invoice_line_items</code> , <code>payments</code> , <code>bank_accounts</code> , <code>owner_statements</code>

DOMAIN	TABLES
Documents	documents , docusign_envelopes
Communications	whatsapp_messages , email_logs , notifications
Workflows	workflow_definitions , workflow_instances

SECTION 07

Row-Level Security (RLS)

PostgreSQL-native row-level security enforces tenant isolation at the database engine level. RLS policies are transparent to the application — queries automatically return only rows belonging to the current organization.

RLS Policy Definition

```
-- rls.sql - Applied to all tenant tables

-- Helper functions read session context variables
CREATE FUNCTION get_current_org_id() RETURNS uuid AS $$
  SELECT NULLIF(current_setting('app.current_org_id', true), '')::uuid;
$$ LANGUAGE sql STABLE;

-- Example: Properties table policy
CREATE POLICY properties_isolation ON "properties"
  FOR ALL TO public
  USING ("organizationId" = get_current_org_id())
  WITH CHECK ("organizationId" = get_current_org_id());
```

Runtime Context Injection

```
// client.ts - Sets tenant context before every query
export async function runInTenantContext<T>(orgId, userId, fn) {
  return await prisma.$transaction(async (tx) => {
    // SET LOCAL scopes these variables to this transaction ONLY
    await tx.$executeRawUnsafe(
      `SET LOCAL app.current_org_id = '${orgId}';`
    );
    await tx.$executeRawUnsafe(
      `SET LOCAL app.current_user_id = '${userId}';`
    );
    return await fn(tx);
  });
}
```

- ✓ **SET LOCAL guarantees:** Session variables are scoped to the transaction only. They are automatically discarded when the transaction ends, preventing context leakage between concurrent requests.

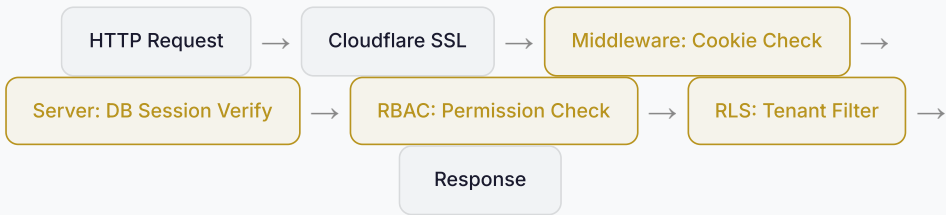
Route & Middleware Protection

Two-layer route protection: lightweight middleware for cookie-presence gating, plus server-side DB verification in route handlers and server components.

Middleware Route Classification

ROUTE PATTERN	ACCESS LEVEL	PROTECTION
/login , /register	Public	No auth required
/api/auth/login , /api/auth/register	Public API	No auth required
/api/webhooks/*	External	Webhook signature verification
/api/bookings/engine	Public API	No auth (public booking engine)
/dashboard/*	Authenticated	Cookie check → DB verify → RBAC
/api/* (all other)	Authenticated API	Cookie check → 401 if missing

Request Flow Through Security Layers



Static Asset Exclusions

```
// Middleware matcher - excludes static files from auth checks
matcher: ['/(?!_next/static|_next/image|favicon.ico|.*\\.png|.*\\.svg).*/']
```

- `_next/static` — Webpack bundles, CSS
- `_next/image` — Optimized image pipeline

- ***.png , *.svg** — Public images and logos

SECTION 09

Data Encryption & Hashing

A summary of all cryptographic operations used across the platform, their algorithms, and the data they protect.

Encryption & Hashing Matrix

DATA	ALGORITHM	STRENGTH	PURPOSE
User Passwords	bcrypt	10 salt rounds (~100ms)	One-way hash — cannot be reversed
Session Tokens	SHA-256	256-bit digest	One-way hash for DB storage (plaintext in cookie only)
Session Token Generation	crypto.randomBytes	256-bit (32 bytes)	CSPRNG — cryptographically secure random
MFA TOTP Secrets	AES-256-GCM	256-bit key, 96-bit IV, auth tag	Reversible encryption — must decrypt for TOTP verify
Transport	TLS 1.3	Via Cloudflare	All data in transit is encrypted
Database Connection	SSL/TLS	sslmode=require	Encrypted connection to Neon PostgreSQL

SECTION 10

Audit Logging

Immutable audit trail records every significant action. Required for UAE regulatory compliance and security incident investigation.

Audit Log Record Structure

FIELD	EXAMPLE	PURPOSE
action	create , update , delete , login	What was done

FIELD	EXAMPLE	PURPOSE
entityType	property , booking , journal_entry	Which entity type
entityId	UUID	Specific record affected
oldValues	JSON snapshot	State before the change
newValues	JSON snapshot	State after the change
userId	UUID	Who performed the action
ipAddress	185.x.x.x	Client IP (forensics)
userAgent	Browser string	Device identification

⚠ **Immutability:** Audit log records are append-only. There is no `UPDATE` or `DELETE` API for audit logs. Older records are retained indefinitely for compliance.

SECTION 11

Cookie & Transport Security

Session cookies are configured with defense-in-depth flags to prevent common web attacks.

Cookie Configuration

```
response.cookies.set( session_id', sessionToken, {
  httpOnly: true,      // Cannot be read by JavaScript (XSS protection)
  secure:   true,      // Only sent over HTTPS (production)
  sameSite: 'strict',  // Not sent with cross-origin requests (CSRF protection)
  path:     '/',       // Available to all routes
  expires:  expiresAt, // 7-day expiration
});
```

Attack Mitigations

ATTACK VECTOR	MITIGATION	FLAG
Cross-Site Scripting (XSS)	Cookie invisible to <code>document.cookie</code>	<code>HttpOnly</code>
Man-in-the-Middle (MitM)	Cookie only sent over HTTPS	<code>Secure</code>
Cross-Site Request Forgery (CSRF)	Cookie not sent with cross-origin requests	<code>SameSite=Strict</code>
Session Fixation	New token generated on every login	Token rotation
Database Breach	Only SHA-256 hash stored, not plaintext	Token hashing

SECTION 12

Environment Separation

Strict environment separation ensures that development, staging, and production are fully isolated with independent secrets and databases.

Environment Configuration

VARIABLE	PURPOSE	ROTATION
DATABASE_URL	PostgreSQL connection (SSL required)	Per environment
MFA_ENCRYPTION_KEY	AES-256 key for TOTP secrets	Never committed to VCS
WHATSAPP_API_TOKEN	Meta Business API token	Rotated quarterly
DOCUSIGN_INTEGRATION_KEY	DocuSign OAuth credentials	Per environment
SENTRY_DSN	Error monitoring endpoint	Per environment

 **Critical Rule:** `.env` files are listed in `.gitignore` and must NEVER be committed to version control. Production secrets are managed via the hosting provider's environment variable UI (Vercel / Cloudflare).

Database Connection Security

```
# .env - SSL required for all database connections
DATABASE_URL="postgresql://user:pass@host/db?sslmode=require&connect_timeout=15"
```